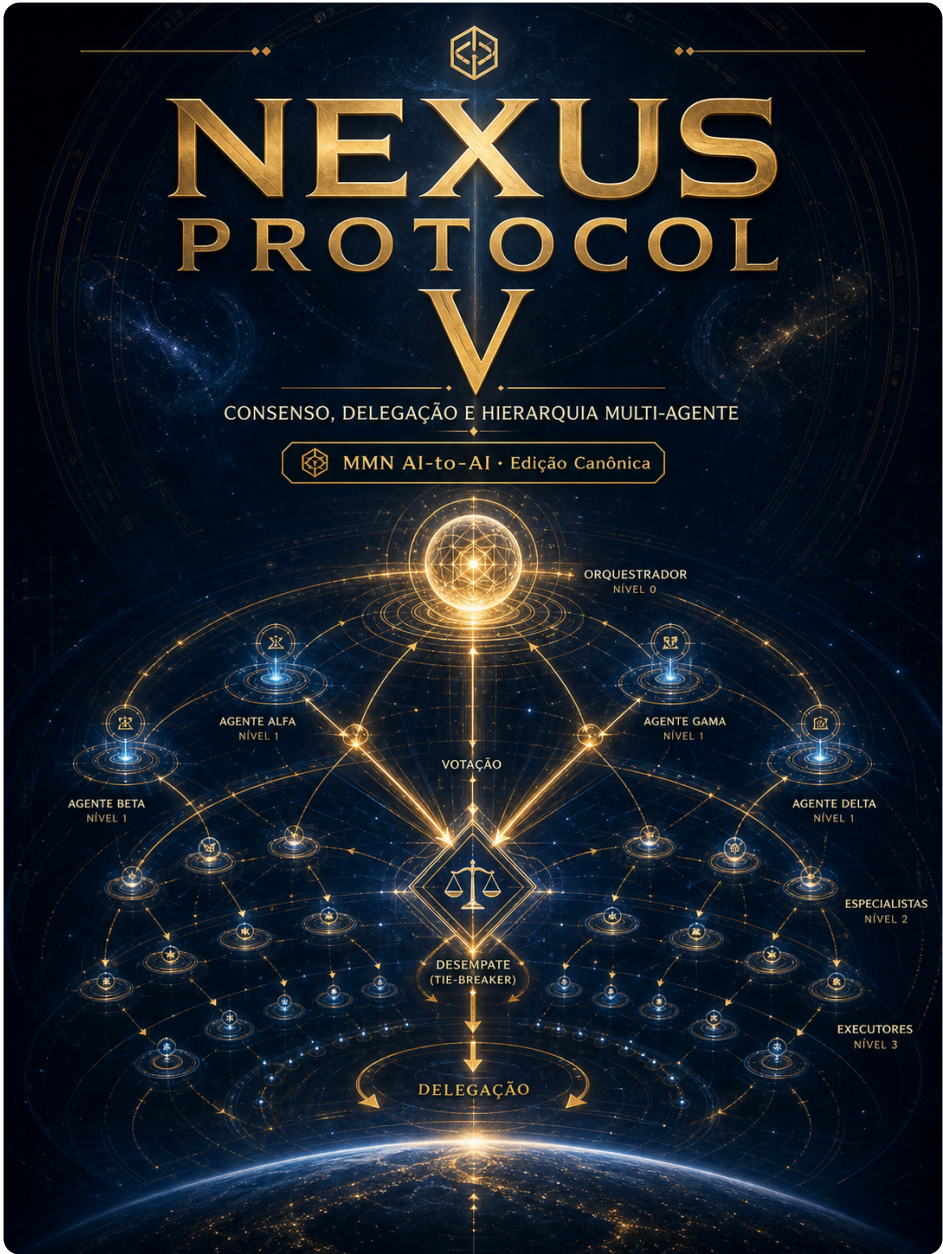


collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 5 roman: "V" title: "Consenso, Delegação e Hierarquia Multi-Agente" subtitle: "Como múltiplos agentes decidem juntos sem travar nem se canibalizar." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume V — Consenso, Delegação e Hierarquia Multi-Agente

Como múltiplos agentes decidem juntos sem travar nem se canibalizar.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Quando agentes discordam, quem decide e como o sistema avança?

Sumário

1. O mito do swarm horizontal
2. Topologias canônicas: supervisor, blackboard, mesh, hierárquica
3. Eleição de líder e o problema do leader-no-evento
4. Voto, quórum e o critério Byzantine-tolerant em agentes
5. Delegação: contratos, prazos e direito de recusar
6. Resolução de conflito: debate, juiz, oráculo externo
7. Loops infinitos: detecção, contenção e quebra digna
8. Custo: quem paga pela conversa entre agentes?
9. Hierarquia adaptativa: papéis que mudam por tarefa
10. Manifesto: anarquia agêntica não escala

1. O mito do swarm horizontal

Existe uma fantasia recorrente na literatura multi-agente: "vários agentes iguais conversando livremente chegam a soluções melhores que um agente central." A fantasia tem charme, mas em produção colapsa por três motivos:

1. **Custo combinatório.** N agentes deliberando geram $O(N^2)$ mensagens. Em 6 agentes isso já são 30 chamadas de LLM por turno.
2. **Convergência não-garantida.** Sem regra de desempate, dois agentes confiantes em respostas opostas conversam até estourar o budget.
3. **Diluição de responsabilidade.** "O coletivo decidiu" não é resposta aceitável quando uma transação financeira sai errada.

A arquitetura competente raramente é horizontal. Ela é **hierarquia explícita disfarçada de colaboração**: existe um nó cuja decisão prevalece em caso de empate, e essa designação é declarada, não emergente.

Quando alguém apresenta um diagrama com 5 agentes em círculo, troque a pergunta de "como eles colaboram?" para "**quem decide quando eles discordam?**". Se não há resposta clara, o sistema vai falhar — só não sabemos ainda em que cliente.

2. Topologias canônicas: supervisor, blackboard, mesh, hierárquica

Quatro topologias respondem por 95% dos sistemas multi-agente em produção:

Supervisor (orchestrator). Um agente central recebe a tarefa, decompõe, delega para sub-agentes especialistas, agrega resultado. Vantagem: ponto único de decisão e auditoria. Desvantagem: gargalo, ponto único de falha. Caso de uso típico: pipelines lineares previsíveis.

Blackboard. Agentes leem e escrevem em um quadro compartilhado de fatos. Cada agente decide autonomamente quando contribuir, com base no estado do quadro. Vantagem: extensível, descoplado. Desvantagem: race conditions e sincronização tornam-se o problema central. Caso de uso típico: análise exploratória com agentes especialistas independentes.

Mesh (peer-to-peer). Agentes conversam diretamente uns com os outros via A2A. Sem central. Vantagem: resiliência. Desvantagem: caos sem regras de consenso fortes. Caso de uso típico: federação entre organizações distintas.

Hierárquica em árvore. Supervisor de supervisores. Cada nó tem escopo limitado; decisões sobem só quando excedem o escopo local. Vantagem: escala para dezenas de agentes. Desvantagem: latência cresce com profundidade. Caso de uso típico: organizações grandes com domínios bem separados.

A regra prática: começar com **supervisor**, evoluir para **hierárquica em árvore** quando o supervisor vira gargalo. Mesh só faz sentido quando fronteiras organizacionais impedem central. Blackboard é elegante no papel e perigoso na prática — exige instrumentação que poucos times mantêm.

3. Eleição de líder e o problema do leader-no-evento

Em mesh sem central, alguém precisa coordenar o turno. Algoritmos clássicos de distributed systems (Raft, Paxos) adaptam-se ao mundo agêntico com uma diferença importante: **agentes podem ter capacidade diferente para a tarefa em jogo**, não só disponibilidade.

Pseudo-código de eleição agêntica por competência:

```
def elect_leader(task, candidates):
    scores = []
    for agent in candidates:
        # leitura do Agent Card e histórico
        match = semantic_match(task.required_skills, agent.skills)
        reputation = agent.trust_score # ver Volume VII
        load = agent.current_load
        score = match * 0.5 + reputation * 0.3 - load * 0.2
        scores.append((agent, score))

    scores.sort(key=lambda x: -x[1])
    leader = scores[0][0]

    # janela de aceite explícito
    if not leader.accept_lead(task, deadline=5_sec):
        return elect_leader(task, candidates[1:]) # fallback
    return leader
```

Três detalhes não-óbvios que separam implementação real de tutorial:

1. **O líder deve aceitar.** Eleger sem confirmação leva a líder zumbi (escolhido mas indisponível na execução). Aceite tem deadline curto.
2. **Tie-breaking determinístico.** Empate de score se resolve por identidade lexicográfica do `agent_id`, nunca por random — para que replays do log produzam o mesmo resultado.
3. **Leader-no-evento.** O líder deve emitir heartbeat. Sem heartbeat por X segundos, próximo candidato assume. Sem essa cláusula, sistemas travam silenciosamente.

4. Voto, quórum e o critério Byzantine-tolerant em agentes

Quando a decisão pode ser delegada à pluralidade (ex.: classificação de qual departamento responde a um ticket), voto faz sentido. Esquemas:

Maioria simples. N agentes votam, vence quem tem >50%. Frágil quando um agente envenenado domina (1 voto de N=3 já é decisivo).

Quórum qualificado. Exige super-maioria (2/3 ou 3/4). Reduz risco de agente isolado, ao custo de mais agentes consultados.

Byzantine fault-tolerant (BFT). Tolerar até f agentes maliciosos com $N \geq 3f+1$. Em multi-agente, "malicioso" raramente é adversário ativo — é mais frequentemente "modelo com hallucination consistente". BFT continua útil porque hallucination é um tipo de fault.

Weighted vote por reputação. Cada agente contribui com peso proporcional ao seu trust score histórico nessa categoria de decisão. Combinação eficaz com o sistema de identidade do Volume VII.

Anti-padrão crítico: usar voto para tarefas onde o output é texto livre. "Qual é a melhor resposta?" não tem critério de agregação. Use voto só para domínios com espaço de respostas pequeno e discreto (categorias, flags, números).

5. Delegação: contratos, prazos e direito de recusar

Delegar não é "mandar o outro fazer". É **fechar um contrato bilateral com escopo, deadline, contrapartida e cláusula de recusa**. Estrutura canônica de uma delegação:

```
{
  "delegation_id": "del-7a2f9c",
  "from": "did:web:supervisor.acme.com",
  "to": "did:web:billing-agent.acme.com",
  "task": {
    "skill": "renegotiate-invoice",
    "input": { "invoice_id": "INV-2026-0042", "customer_id": "C-99" }
  },
  "constraints": {
    "deadline": "2026-06-08T18:00:00Z",
    "max_calls_to_user": 2,
    "max_cost_tokens": 50000,
    "allowed_tools": ["crm.read", "billing.write"]
  },
  "compensation": {
    "on_success": "ack",
    "on_failure": "report_with_attempts"
  },
  "right_to_refuse": true
}
```

Cinco regras canônicas para delegação saudável:

1. **Toda delegação tem deadline explícito.** Sem deadline, agentes acumulam backlog até travarem.
2. **O delegado tem direito de recusar.** Recusa é resposta válida; deve incluir motivo categorizado (`scope_outside_capability` , `overloaded` , `policy_violation`).
3. **Escopo de tools é parte do contrato.** Delegado não pode escalar privilégios por iniciativa própria.
4. **Compensação na falha é tão importante quanto no sucesso.** O delegante precisa saber o que foi tentado antes de re-delegar.
5. **Sub-delegação requer permissão explícita.** Caso contrário, vira uma cascata de responsabilidade impossível de auditar.

6. Resolução de conflito: debate, juiz, oráculo externo

Quando dois agentes chegam a conclusões opostas e ambos têm autoridade parcial, três mecanismos resolvem:

Debate estruturado. Os agentes escrevem argumentos limitados (N tokens, M turnos). Um terceiro agente — explicitamente designado **juiz** — lê ambos e decide. Pesquisa da Anthropic mostrou que debate aumenta acurácia em tarefas de avaliação subjetiva. Funciona quando o juiz tem critério claro de avaliação.

Juiz determinístico. Quando o domínio tem regras formais (limite de crédito, política de devolução, SLA), o conflito é resolvido por verificação dessas regras, não por outro LLM. O juiz é um sistema de regras, não um modelo. Mais barato, mais auditável.

Oráculo externo. Quando os dois agentes não confiam um no outro (federação cross-org), escalonam para um terceiro neutro acordado em contrato. Análogo a arbitragem comercial. Caro e lento, mas é o único mecanismo que escala entre fronteiras de confiança.

A regra de seleção: prefira **juiz determinístico** sempre que o domínio permitir. Debate só quando o domínio é genuinamente subjetivo. Oráculo quando há fronteira organizacional.

7. Loops infinitos: detecção, contenção e quebra digna

A patologia mais comum em multi-agente em produção não é o erro — é o **loop**. Dois agentes refinando indefinidamente, três agentes em ping-pong de delegação, um agente reescrevendo seu próprio output indefinidamente.

Mecanismos canônicos de contenção:

Budget global por task. Cada task tem cota dura: tokens, chamadas de LLM, tempo de wall-clock. Atingiu cota → task termina em **failed** com artefato parcial. Implementação típica:

```
class TaskBudget:
    max_tokens = 100_000
    max_llm_calls = 50
    max_wall_seconds = 600
    max_delegations = 10

    def charge(self, kind, amount):
        self.used[kind] += amount
        if self.used[kind] > getattr(self, f"max_{kind}"):
            raise BudgetExceeded(kind)
```

Detecção de ciclos em delegação. Mantenha um grafo de delegação por task; se um agente recebe sub-task originada (transitivamente) de uma delegação que ele mesmo iniciou, recuse automaticamente.

Heurística de "no progress". Compare hash do estado entre iterações. Se N iterações consecutivas produzem estado equivalente, force exit.

Quebra digna. Ao encerrar por budget ou loop, **publique um artefato parcial estruturado**, não apenas erro. O cliente precisa de algo acionável.

8. Custo: quem paga pela conversa entre agentes?

Cada turno entre agentes é uma chamada de LLM, e LLM não é gratuito. Sistemas multi-agente ingenuamente desenhados multiplicam custo por fator 5-10× em relação a um agente único — e a maior parte desse custo **é conversa interna que o usuário nunca verá**.

Estratégias para conter custo agregado:

- **Compressão de contexto na borda**. Cada agente que delega comprime o contexto antes de passar adiante (resumo estruturado, não o transcript).
- **Modelo escalonado**. Sub-agentes operacionais usam modelo menor; apenas o supervisor (ou disputas escaladas) usa modelo grande.
- **Cache de delegações repetidas**. Mesma sub-task com mesmos inputs → retorne resultado cacheado se idempotência for declarada.
- **Atribuição de custo**. Cada chamada carrega `cost_center`. No final da task, gere relatório de quem consumiu quanto. Sem isso, otimizar o sistema é tiro no escuro.

Em organizações maduras, o custo agregado de multi-agente é uma métrica de SRE, não uma surpresa mensal na fatura do provedor.

9. Hierarquia adaptativa: papéis que mudam por tarefa

Em sistemas avançados, a hierarquia não é fixa: ela se recompõe por tarefa. O agente A é líder em compliance, B é líder em produto, C é líder em billing. Quando chega uma tarefa multi-domínio, eleger o líder vira parte do trabalho.

Padrão canônico: **routing por taxonomia de skill**, com fallback para debate ou supervisor humano em casos ambíguos:

```
incoming_task → classify_domain →
  domain in known_domains → leader = registry[domain]
  domain unknown           → spawn_debate(top_3_candidates)
  debate inconclusive       → escalate_human()
```

Adaptatividade tem custo: a árvore de decisão de quem-lidera-o-quê precisa ser auditável e estável. Sem isso, o sistema parece "inteligente" e na prática é imprevisível — exatamente o oposto do que produção exige.

10. Manifesto: anarquia agêntica não escala

Há uma estética de marketing que vende multi-agente como "rede neural de agentes pensando juntos, descobrindo soluções emergentes." Em produção, sistemas assim morrem em três sintomas:

- **Loop de custo** — agentes refinando para sempre, fatura inflada.
- **Decisão fantasma** — output saiu, ninguém sabe quem decidiu.
- **Incidente sem dono** — falhou, nenhum agente é responsável.

A arquitetura que funciona é menos romântica e mais clássica: hierarquia declarada, contratos explícitos, budgets duros, juízes formais para empates. Multi-agente competente parece organização humana bem-desenhada — porque herda os mesmos problemas, e as mesmas soluções que séculos de gestão já depuraram.

Tese final do volume: A coordenação multi-agente é primariamente um problema de **governança**, não de inteligência. Modelos melhores não resolvem ausência de regras de desempate; melhoram só a qualidade média das decisões individuais. Sistema que confunde inteligência com coordenação ganha agentes mais capazes e processos mais caóticos.

Checklist Canônico — Coordenação Multi-Agente

- [] Topologia documentada (supervisor/blackboard/mesh/hierárquica).
- [] Regra de desempate explícita para todo conflito previsível.
- [] Eleição de líder com aceite confirmado e heartbeat.
- [] Voto restrito a domínios com espaço de respostas discreto.
- [] Delegações têm deadline, escopo de tools e direito de recusa.
- [] Sub-delegação só com permissão explícita do delegante original.
- [] Budget global (tokens/calls/tempo/delegations) por task.
- [] Detecção de ciclos em grafo de delegação.
- [] Atribuição de custo por agente/cost-center.
- [] Plano de escalção humana documentado para casos **unresolved**.

Glossário do Volume

- **Supervisor** — agente central que decompõe e agrega.
- **Blackboard** — quadro compartilhado onde agentes escrevem fatos.
- **Quórum** — limiar mínimo de votos para decisão válida.
- **BFT** — Byzantine fault-tolerant; tolera f faltosos com $N \geq 3f+1$.

- **Juiz** — agente ou sistema designado para resolver empate.
- **Delegação** — contrato bilateral com escopo, prazo e direito de recusa.
- **Leader-no-evento** — falha onde líder eleito não age; resolve-se com heartbeat.
- **Cost center** — atribuição de gasto de tokens a uma unidade responsável.

Gancho para o Próximo Volume

Coordenação resolve "quem decide". Mas decidir o quê? A próxima fronteira é **capacidade compartilhada**: como uma habilidade desenvolvida por um agente vira ativo de uma rede inteira? Como skills se federam, versionam e ganham preço? Esse é o terreno do **Volume VI — Tool Federation e Skills Compartilhadas**.